AI Applications in Research

Concepts, Workflows, and the Use of Claude Code
in Quantitative Social Science

Compiled for the Scott Cunningham newsletter routine

9 September 2026

Abstract

This document summarizes the main concepts behind using AI — and in particular *Claude Code*, an agentic command-line coding assistant — in empirical research. It draws on Scott Cunningham's ongoing "Claude Code" series on *Scott's Mixtape* Substack, which documents in real time how an applied economist integrates AI agents into causal-inference work. The emphasis throughout is on **reproducibility**, **cross-implementation auditing**, and the practical reshaping of the research workflow, alongside the new risks these tools introduce.

Contents

1 What is Claude Code and why researchers care **1**

2 Core concepts of AI-assisted research **2**
2.1 Multi-agent code auditing (replication as verification) 2
2.2 Resurrecting and extending old projects 2
2.3 Reproducibility and documentation 2
2.4 New problems created by AI, even when it saves time 2

3 The research workflow, step by step **2**

4 Benefits and risks at a glance **3**

5 Latest newsletter summary **3**

6 Takeaways **4**

---

1 What is Claude Code and why researchers care

Claude Code is a terminal-based "agentic" assistant: rather than answering a single prompt, it reads a project directory, plans multi-step work, edits files, runs code, inspects the output, and iterates toward a goal. For a researcher this changes the unit of interaction from "write me a function" to "*take this project and do X*." Cunningham's recurring framing is that the interesting question is not Claude Code in the abstract or from a software engineer's point of view, but its usefulness for **applied empirical work** — econometrics, replication, and causal inference.

Key properties that matter for research:

- **Repository awareness.** The agent operates over a whole project (data, scripts, notes), not an isolated snippet, so it can resurrect and extend old codebases — e.g., taking a ten-year-old project and reconstructing its workflow.
- **Multi-language fluency.** It moves across Stata, R, and Python in a single session, which is central to the auditing use cases below.
- **Autonomy with a human in the loop.** It proposes and executes steps while the researcher supervises, corrects, and documents — Cunningham calls the resulting practice "my workflow" rather than pure automation.

2 Core concepts of AI-assisted research

2.1 Multi-agent code auditing (replication as verification)

The signature idea in the recent series is using *multiple independent agents* to replicate the same analytical pipeline from scratch across different languages and packages, then checking whether the deterministic outputs agree. The worked example is difference-in-differences estimated over:

- **Three languages:** Python, R, and Stata.
- **Five packages:** two Stata packages (`csdid`, `csdid2`), one R package (`did`), and two Python packages (`diff-diff`, `differences`), centered on the Callaway and Sant'Anna estimator.

Because non-random quantities (number of observations, means, R^2, point estimates) should be identical across correct implementations, disagreement flags a bug in one of them. AI turns an otherwise prohibitively expensive cross-package replication into a routine **robustness and validation check**.

2.2 Resurrecting and extending old projects

Agents lower the cost of returning to dormant work: pointing Claude Code at a legacy directory to rebuild the pipeline and then extend it with newer methods (for instance, moving an old binary-treatment paper toward *continuous* difference-in-differences).

2.3 Reproducibility and documentation

The workflow externalizes tacit knowledge: the agent's plan, commands, and diffs become an auditable record. Documenting "what works and what doesn't" in public is itself part of Cunningham's method — the newsletter is a live lab notebook.

2.4 New problems created by AI, even when it saves time

A recurring, honest theme: AI solves old problems while *creating new ones*, sometimes without even reducing total time spent. Examples include over-trust in plausible-but-wrong output, loss of the friction that used to force careful thinking, and the effort of verifying agent work. The multi-agent auditing pattern is partly a response to exactly this risk.

3 The research workflow, step by step

A representative Claude-Code-in-research loop:

1. **Point the agent at a directory** — an existing project or a fresh replication target.
2. **Ask for a plan**, then supervise as the agent reads data and code and proposes steps.
3. **Fan out across implementations** — have separate agents build the same estimand in Stata, R, and Python.
4. **Diff the deterministic outputs** to confirm agreement; investigate any mismatch as a candidate bug.
5. **Extend the analysis** with newer estimators or specifications.
6. **Document** the session so the result is reproducible and the reasoning is inspectable.

4 Benefits and risks at a glance

+ Cross-language, cross-package replication becomes cheap enough to do routinely.
+ Old projects can be revived and modernized quickly.
+ Workflows are more legible and reproducible when the agent's actions are recorded.
− Confident-but-wrong output demands active verification.
− Time savings are not guaranteed — new verification and supervision costs appear.
− Skills can erode if researchers offload reasoning wholesale rather than supervising.

5 Latest newsletter summary

Most recent identifiable post: *"Claude Code 24: Multiple Agents Auditing Your Diff-in-Diff Code (Part 1)"*, from *Scott's Mixtape* Substack.[1]

Main topics.

- Using several AI agents in parallel to audit difference-in-differences code.
- Cross-implementation replication as a correctness check.
- Continuation of the practical "Claude Code for quantitative social scientists" series begun in mid-December 2025.

Key points (one line each).

- Multiple agents each rebuild the same diff-in-diff pipeline independently, from scratch.
- The comparison spans Stata (`csdid`, `csdid2`), R (`did`), and Python (`diff-diff`, `differences`).

[1] Live retrieval of the Substack was not possible from this environment (network egress to `causalinf.substack.com` is blocked), so this summary is compiled from public search results rather than the full post body. As the series is ongoing, a still-newer installment may exist.

- Non-random outputs should match across all correct implementations; divergence signals a bug.
- The Callaway and Sant'Anna estimator is the focal method.
- "Part 1" signals a multi-post treatment of the auditing pattern.

Links, resources, and calls to action.

- Claude Code series hub: https://causalinf.substack.com/s/claude-code
- This post: https://causalinf.substack.com/p/claude-code-24-multiple-agents-auditing
- Cunningham's tooling repository (*Mixtape Tools*): https://github.com/scunning1975/mixtapetools
- Implicit call to action: subscribe to *Scott's Mixtape* and try the multi-agent audit on your own diff-in-diff code.

6 Takeaways

1. Agentic AI shifts research assistance from snippets to whole-project execution.
2. The most valuable pattern for empiricists is *redundant, cross-implementation replication* used as verification.
3. Reproducibility improves when the agent's plan and actions are recorded.
4. Verification and supervision are non-negotiable — AI creates new failure modes even as it removes old ones.

Sources: *Scott's Mixtape* Substack (https://causalinf.substack.com); Claude Code series hub (https://causalinf.substack.com/s/claude-code). Compiled from public search results; the newsletter body could not be fetched directly from this environment.